ВИВЧЕННЯ C# ПРОСТО ДЛЯ ПОЧАТКІВЦІВ

ПРОГРАМУВАННЯ НА C# ВІД НУЛЯ ДО МАЙСТЕРНОСТІ

(ВСТУП)

ПАТРІК ФЕЛІСІЯ

Переклад українською мовою

C# від нуля до майстерності

(Вступ)

Покрокова інструкція з вивчення C#.

Зміст

Титульна сторінка

Програмування на C# від нуля до майстерності (Вступ)

1 Вступ до програмування на C#

2 Подяка

Також від Патріка Фелісії

C# від нуля до майстерності (Вступ)

Авторське право © 2020 Патрік Фелісія

Усі права захищені. Жодна частина цієї книги не може бути відтворена, збережена в пошукових системах або передана в будь-якій формі чи будь-якими засобами без попереднього письмового дозволу видавця (Патріка Фелісії), за винятком коротких цитат, вбудованих у критичні статті або огляди.

Було докладено всіх зусиль при підготовці цієї книги для забезпечення точності представленої інформації. Однак інформація, що міститься в цій книзі, продається без гарантії, явної чи неявної. Ні автор, ні його дилери та дистриб'ютори не несуть відповідальності за будь-які збитки, спричинені або нібито спричинені прямо чи опосередковано цією книгою.

Перше видання: лютий 2020

Видано Патріком Фелісією

Подяки

Автор: Патрік Фелісія.

Патрік Фелісія — викладач та дослідник у Технологічному інституті Вотерфорда, де він викладає та керує студентами бакалаврату та магістратури. Він отримав ступінь магістра з мультимедійних технологій у 2003 році та ступінь доктора комп'ютерних наук у 2009 році в Університетському коледжі Корка, Ірландія. Він опублікував кілька книг та статей про використання відеоігор в освітніх цілях. Патрік також є головним редактором Міжнародного журналу ігрового навчання (IJGBL) та директором конференції Ірландського симпозіуму з ігрового навчання. Він також опублікував книги з програмування на C# через створення ігор.

Підтримка та ресурси для цієї книги

Для цієї книги створено супутній веб-сайт, де ви можете завантажити повний проєкт для цієї книги з усім кодом, щоб ви могли перевірити свій власний код, а також рішення до завдань, наданих у кожному розділі.

Щоб скористатися цим контентом, відкрийте наступне посилання:

http://learncsharpforbeginners.wordpress.com/

Цей сайт також надає вам можливість підписатися на розсилку та отримувати:

- вашу книгу у форматі PDF (БЕЗКОШТОВНА копія), щоб ви могли відкрити її на будь-якому комп'ютері.
- регулярні оновлення та підручники з C#.
- купони на знижки понад 30% на деякі книги автора.

Ця книга присвячена Матісу та Гелені

Передмова

Ця книга є частиною серії під назвою «C# від нуля до майстерності». У цій серії книг ви маєте можливість вивчити програмування на C# з нуля та стати професіоналом у цій популярній мові програмування.

У цій книзі під назвою «C# від нуля до майстерності (Вступ)» ви дізнаєтесь, як швидко розпочати роботу з C#; ви навчитеся використовувати змінні, методи, умовні оператори, а також напишете свій власний код.

Що вам потрібно для використання цієї книги

Для виконання проєкту, представленого в цій книзі, вам потрібне лише інтегроване середовище розробки (IDE), таке як Visual Studio Community.

Що стосується комп'ютерних навичок, уся інформація, представлена в цій книзі, не передбачає попереднього досвіду програмування від читача. Хоча ця книга містить програмування, вас буде направлятися крок за кроком. Тому наразі вам потрібно лише вміти виконувати звичайні комп'ютерні завдання, такі як набір тексту, відкриття та збереження файлів.

Для кого ця книга

Якщо ви можете відповісти «так» на всі ці запитання, то ця книга для вас:

- Ви абсолютний початківець у програмуванні на C#?
- Ви хотіли б швидко освоїти концепції C#?
- Хоча ви, можливо, вже мали деякий досвід з C#, чи хотіли б ви глибше зануритися в C#?

Якщо ви можете відповісти «так» на всі ці запитання, то ця книга не для вас:

- Ви вже можете вільно програмувати на C#?
- Ви шукаєте довідкову книгу з C#?
- Ви досвідчений (або принаймні просунутий) програміст на C#?

Як ви будете навчатися з цієї книги

Оскільки всі студенти навчаються по-різному та мають різні очікування від курсу, ця книга розроблена так, щоб усі читачі знайшли зручний для них режим навчання. Тому вона включає наступне:

- Список навчальних цілей на початку кожного розділу, щоб читачі мали уявлення про навички, які будуть розглянуті.
- Кожен розділ включає огляд діяльності, що розглядається.
- Багато завдань є покроковими, а учні також мають можливість поглибити навчання та навички вирішення проблем через завдання, запропоновані в кінці кожного розділу.
- Кожен розділ завершується вікториною та завданнями, через які ви можете перевірити свої навички та знання.
- Книга зосереджується на основних навичках, які вам потрібні.
- Код вводиться поступово та детально пояснюється.
- Ви також отримуєте доступ до кількох відео, які допоможуть вам у навчанні.

Формат кожного розділу та умовні позначення

У цій книзі, для полегшення читання та навчання, використовуються форматування тексту та значки для виділення частин наданої інформації та для зручності читання книги.

Спеціальні примітки

Кожен розділ включає розділи з ресурсами, щоб ви могли поглибити своє розуміння та майстерність C#; вони включають:

- Вікторину для кожного розділу: зазвичай вона включає 10 запитань, що перевіряють ваші знання тем, розглянутих у розділі. Відповіді надані на супутньому веб-сайті.
- Контрольний список: складається з 5-10 ключових концепцій та навичок, з якими ви повинні бути комфортні перед переходом до наступного розділу.
- Завдання: кожен розділ включає розділ з завданнями, де вас просять поєднати свої навички для вирішення конкретної проблеми.

Код відображається наступним чином:

```
public int score;
public string playersName = "Sam";
```

Як найкраще навчатися з цієї книги?

Розмовляйте з друзями про те, що ви робите.

Ми часто думаємо, що розуміємо тему, поки нам не доводиться пояснювати її друзям та відповідати на їхні запитання. Пояснюючи свої різні проєкти, те, що ви щойно вивчили, стане для вас зрозумілішим.

Виконуйте вправи.

Усі розділи включають вправи, які допоможуть вам навчатися на практиці. Іншими словами, виконуючи ці вправи, ви зможете краще зрозуміти теми та отримати практичні навички.

Не бійтеся робити помилки.

Я зазвичай кажу своїм студентам, що помилки — це частина процесу навчання; чим більше помилок ви робите, тим більше можливостей для навчання ви маєте.

Кидайте собі виклик.

Усі розділи включають розділ із завданнями, де ви можете вирішити прийняти конкретний виклик для покращення своєї гри або навичок.

Навчайтеся частинами.

Може бути складно пройти п'ять або шість розділів поспіль. Натомість дайте собі достатньо часу для навчання, рухайтеся у своєму темпі та навчайтеся невеликими порціями (наприклад, по 15-20 хвилин на день). Це дасть вашому мозку час «перетравити» інформацію та допоможе зберегти мотивацію.

Зміст, що охоплюється цією книгою

Розділ 1, Вступ до програмування на C#, надає введення в загальні концепції програмування, включаючи синтаксис C#, змінні та методи. У цьому розділі ви почнете створювати свій перший додаток і комбінувати змінні та методи.

Розділ 2 підсумовує теми, розглянуті в цій книзі, та надає вам більше інформації про наступні кроки.

1. Вступ до програмування на C#

У цьому розділі ми пройдемо вступ до програмування на C# та розглянемо ключові аспекти, які вам знадобляться для ваших програм, включаючи:

- Синтаксис C#.
- Типи та область видимості змінних.
- Методи.

Отже, після завершення цього розділу ви зможете:

- Розуміти ключові концепції, пов'язані з програмуванням на C#.
- Розуміти концепції змінних та методів.
- Створювати та використовувати власні методи.

Рішення коду для цього розділу включені в пакет ресурсів, який ви можете завантажити, дотримуючись інструкцій у розділі «Підтримка та ресурси для цієї книги».

Що таке C#

C# — це об'єктно-орієнтована мова, розроблена Microsoft, яка використовувалася в багатьох випадках для розробки настільних додатків. Оскільки вона є частиною фреймворку .NET, вона також використовується для веб-розробки та додатків. Те, що це об'єктно-орієнтована мова програмування, означає, що програма на C# складається з колекції об'єктів, які взаємодіють один з одним.

C# використовується для багатьох додатків, включаючи створення ігор, особливо з програмним забезпеченням Unity.

Вибір редактора коду

У наступних розділах ви почнете писати свій власний код C#. Для цієї мети вам потрібно буде використовувати редактор коду, а точніше, інтегроване середовище розробки (IDE). Ці IDE включають усі необхідні інструменти для компіляції та виконання вашого коду.

Використання IDE надасть вам простіший спосіб створювати, організовувати та перевіряти ваш код з такими функціями:

- Підсвічування тексту: щоб ваш код було зрозуміліше читати та редагувати.
- Автодоповнення: щоб вам не потрібно було запам'ятовувати назви всіх змінних у вашому коді або назви деяких поширених методів.
- Підсвічування помилок: щоб деякі очевидні помилки виділялися ще до того, як ви протестуєте свій код.

Є багато безкоштовних IDE з їхніми перевагами та недоліками, включаючи: Visual Studio Code, Visual Studio Community, MonoDevelop, NetBeans, #Develop, Rider.

З усіх цих IDE ми будемо використовувати Visual Studio Community для цієї книги (хоча ви можете використовувати будь-яку іншу IDE за бажанням).

Ви можете завантажити Visual Studio Community з наступного сайту:

https://visualstudio.microsoft.com/free-developer-offers/

Встановлення Visual Studio

Будь ласка, завантажте Visual Studio з офіційного сайту та дотримуйтесь інструкцій; ви зможете завантажити та встановити версію для Windows або Mac OS з наступного сайту:

https://visualstudio.microsoft.com/free-developer-offers/

Створення вашого першого додатку

Після цього короткого вступу, настала ваша черга програмувати.

Щоб ви могли експериментувати зі своїм кодом C#, будь ласка, виконайте наступне після встановлення Visual Studio:

- Запустіть VS (Visual Studio).
- У головному вікні натисніть кнопку «New Project» (Новий проєкт).
- У новому вікні виберіть опцію Other > .NET > Console Project (C#).

Вибравши цю опцію, ви вказуєте, що будете створювати проєкт, який буде виконуватися у вікні консолі.

- Натисніть кнопку «Next» (Далі).
- У наступному вікні введіть назву та місце розташування вашого нового проєкту (наприклад, myFirstProject).
- Натисніть кнопку «Create» (Створити).

Після цього Visual Studio створить усі необхідні папки для вашого проєкту, і редактор коду повинен відкритися.

За замовчуванням ви побачите, що деякий код уже було створено для вас:

```
using System;
namespace myFirstProject
{
    class MainClass
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

У попередньому коді:

- Ми оголошуємо, що будемо використовувати бібліотеку (або шлях) під назвою System. Це для того, щоб ми могли використовувати деякі корисні файли (тобто класи), включені в цей простір імен (або бібліотеку).
- Ми оголошуємо простір імен під назвою myFirstProject — ми розглянемо цю концепцію пізніше. Це еквівалент шляху або бібліотеки для деяких класів, які ви будете використовувати.
- Ми оголошуємо клас під назвою MainClass. Як ми побачимо пізніше, ваш додаток складатиметься з колекції класів. Один з цих класів буде класом MainClass, який є точкою входу для вашого додатку.

- Ми також оголошуємо метод під назвою Main. Цей метод буде точкою входу для нашого додатку.
- У цьому методі, і це найважливіша частина наразі, ми записуємо текст "Hello World" у вікно консолі.

Тепер ви можете скомпілювати та запустити цей код, натиснувши чорний трикутник, розташований у верхньому лівому куті вікна.

Ви також можете запустити свій додаток наступним чином: з верхнього меню виберіть: Run > Run With > Custom Configuration... Залиште всі параметри за замовчуванням і натисніть кнопку «Run».

Після цього вікно консолі повинно з'явитися, і текст "Hello World" буде написано та відображено у вікні консолі.

Тепер ми можемо змінити наш код, щоб повідомлення було іншим:

- Закрийте вікно консолі.
- Поверніться до VS та відредагуйте свій код наступним чином:

```
public static void Main(string[] args)
{
    Console.WriteLine("Hello, this is my fist line of code");
}
```

У попередньому коді ми просто змінили повідомлення, що відображається у вікні консолі. Тепер ви можете запустити код і побачити нове повідомлення.

Вступ до синтаксису C#

Коли ви використовуєте C#, ви спілкуєтесь із системою, просячи її виконати дії. Для спілкування з системою ви використовуєте мову або набір слів, пов'язаних синтаксисом, який знають і комп'ютер, і ви. Ця мова складається з ключових слів, ключових фраз та синтаксису, який забезпечує правильне написання та (що важливіше) розуміння інструкцій.

У комп'ютерних науках ця мова повинна бути точною, чіткою, однозначною та з правильним синтаксисом.

Коли ви пишете код C#, ви будете дотримуватися синтаксису; цей синтаксис складатиметься з набору правил, які дозволять вам однозначно передавати ваші інструкції. На додаток до свого синтаксису, C# також використовує класи, і ваші додатки C# за замовчуванням зберігатимуться як класи.

Оператори (Statements)

Коли ви програмуєте на C#, вам потрібно вказати системі виконувати ваші інструкції (наприклад, друкувати інформацію) за допомогою операторів. Оператор — це буквально наказ або щось, що ви просите систему зробити. Наприклад, у наступному рядку коду оператор вказує системі надрукувати повідомлення у вікні консолі:

```
Console.WriteLine("Hello Word");
```

При написанні операторів вам потрібно дотримуватися кількох правил, включаючи:

- Порядок операторів: кожен оператор виконується в тому ж порядку, в якому він з'являється в додатку.

Наприклад, у наступному прикладі код надрукує спочатку «hello», потім «world»:

```
Console.WriteLine("hello");
Console.WriteLine("world");
```

- Оператори розділяються крапкою з комою (тобто крапка з комою в кінці кожного оператора).
- Кілька операторів можуть бути додані в один рядок, якщо вони розділені крапкою з комою.

Наприклад, наступний рядок коду має правильний синтаксис:

```
Console.WriteLine("hello"); Console.WriteLine("world");
```

- Множинні пробіли ігноруються для операторів; проте гарною практикою є додавання пробілів навколо операторів +, -, /, або % для зрозумілості.

```
a = b;
```

- Оператори, які повинні виконуватися разом, можуть бути згруповані за допомогою блоків коду. У C# блоки коду позначаються фігурними дужками ({ або }).

```
{
    Console.WriteLine("hello stranger!");
    Console.WriteLine("today, we will learn about programming");
}
```

Як ми бачили раніше, оператор зазвичай використовує або починається з ключового слова (тобто слова, яке комп'ютер знає). Кожне з цих ключових слів має конкретне призначення, і поширені з них:

- Друк повідомлення у вікні консолі: ключове слово — Console.WriteLine.
- Оголошення змінної: ключове слово залежить від типу оголошуваної змінної (наприклад, int для цілих чисел, string для тексту або bool для булевих змінних).
- Оголошення методу: ключове слово залежить від типу даних, що повертаються методом (наприклад, int, string або void).
- Позначення блоку інструкцій для виконання на основі умови: ключові слова — if та else.
- Вихід з методу: ключове слово — return.

Тепер, коли ви знаєте трохи більше про оператори, давайте змінимо ваш початковий код, щоб включити більше операторів. Додайте наступний код:

```
Console.WriteLine("Hello, this is my fist line of code");
{
    Console.WriteLine("this is my first application");
    Console.WriteLine("This statement and the previous one
        belong to the same block of instructions");
}
```

У попередньому коді ми створили блок інструкцій, що починається відкриваючою фігурною дужкою та закінчується закриваючою фігурною дужкою. Всередині цього блоку ми створили два оператори; обидва оператори закінчуються крапкою з комою.

Коментарі

У C# ви можете використовувати коментарі для пояснення свого коду та для полегшення його читання іншими. Це стає важливим у міру зростання розміру вашого коду; а також важливо, якщо ви працюєте в команді.

Код, який закоментовано, зазвичай не виконується. Є два способи коментування коду в C# — однорядкові та багаторядкові коментарі.

В однорядкових коментарях подвійна коса риска додається на початку рядка або після оператора:

```
//наступний рядок друкує Hello у вікні консолі
Console.WriteLine("Hello");
//наступний рядок оголошує змінну name
string name;
name = "Hello";//встановлює значення змінної name
```

У багаторядкових коментарях будь-який код між символами /* та */ буде закоментований і не буде виконуватися:

```
/* наступні рядки після коментарів надрукують
   повідомлення "hello" у вікні консолі
   потім ми оголосимо змінну name та присвоїмо значення
*/
Console.WriteLine("Hello");
string name;
name = "Hello";
```

Окрім надання пояснень до коду, ви також можете використовувати коментарі для запобігання виконанню частини коду. Це дуже корисно при налагодженні коду. Коментуючи розділи коду та використовуючи процес виключення, ви зазвичай можете швидко знайти проблему. Цей процес часто називають дихотомією.

Давайте використаємо коментарі в нашому додатку. Змініть код наступним чином:

```
public static void Main(string[] args)
{
    //Console.WriteLine("Hello, this is my fist line of code");
    Console.WriteLine("Hello, this is my second line of code");
```

Збережіть код та запустіть додаток; ви побачите, що повідомлення "Hello this is my first line of code" було замінено на "Hello this is my second line of code".

Ми також можемо використати багаторядкові коментарі:

```
Console.WriteLine("Hello, this is my second line of code");
{
    /*Console.WriteLine("this is my first application");
    Console.WriteLine("This statement and the previous one
        belong to the same block of instructions");*/
}
```

Якщо ви збережете код та запустите додаток, єдиним повідомленням, що відобразиться, буде "Hello this is my second line of code".

Змінні

Тепер, коли ви знаєте трохи більше про оператори та коментарі, ми почнемо вивчати більше про змінні, які є досить важливими в C#, оскільки ви, ймовірно, будете використовувати їх у більшості своїх програм для виконання обчислень або для роботи з текстом.

Отже, що таке змінна?

Змінну можна порівняти з контейнером, який містить значення, яке може змінюватися з часом. При використанні змінних нам зазвичай потрібно: (1) оголосити змінну, вказавши її тип, (2) присвоїти значення цій змінній, і (3) можливо комбінувати цю змінну з іншими змінними за допомогою операторів:

```
int myAge;//ми оголошуємо змінну myAge
myAge = 20;// ми встановлюємо змінну myAge на 20
myAge = myAge + 1; //ми додаємо 1 до змінної myAge
```

У попередньому прикладі ми оголосили змінну під назвою myAge, і її тип — int (ціле число). Ми зберігаємо значення 20 в цій змінній, а потім додаємо 1.

Зверніть увагу, що в C# змінна оголошується за допомогою її типу, за яким слідує її назва.

Також зверніть увагу, що оператор = є оператором присвоєння; іншими словами, він призначений для присвоєння значення змінній і не повинен розумітися в строгому алгебраїчному сенсі.

Для полегшення кодування C# ви можете оголосити кілька змінних одного типу в одному операторі:

```
int v1, v2, v3;
int v4=4, v5=5, v6=6;
```

При використанні змінних є кілька речей, які потрібно визначити:

- Назва змінної: змінній зазвичай дається унікальна назва, щоб її можна було легко та однозначно ідентифікувати. Назву змінної зазвичай називають ідентифікатором.
- Тип змінної: змінні можуть зберігати кілька типів даних, включаючи числа (наприклад, цілі числа, double або float), текст (наприклад, рядки або символи), булеві значення (true або false), масиви або об'єкти.
- Оголошення змінної: змінні потрібно оголошувати, щоб система знала, до чого ви звертаєтесь.
- Область видимості змінної: змінна може бути доступна в конкретних контекстах, що залежать від того, де в додатку змінна була спочатку оголошена.
- Рівень доступності: як ми побачимо пізніше, програма C# складається з класів; для кожного з цих класів методи та змінні всередині можуть бути доступні залежно від їхніх рівнів доступності.

Поширені типи змінних включають:

- String: те саме, що текст.
- Int: ціле число (1, 2, 3 тощо).
- Boolean: true або false.

- Float: з дробовим значенням (наприклад, 1.2f, 3.4f тощо).
- Arrays (масиви): група змінних одного типу.

Використання змінних

У цьому розділі ми почнемо використовувати змінні та комбінувати їх за допомогою операторів. Отже, давайте почнемо. Додайте наступний код:

```
int dateOfBirth = 1990;
int currentYear = 2017;
int age = currentYear - dateOfBirth;
Console.WriteLine("My age is " + age);
```

У попередньому коді:

- Ми оголосили дві змінні dateOfBirth та currentYear.
- Ми потім оголосили змінну під назвою age.
- Ми виконали віднімання між змінною currentYear та змінною dateOfBirth, а потім зберегли результат у змінній age.
- Нарешті, ми друкуємо текст "My age is ", за яким слідує значення змінної age.

Зверніть увагу, що додавання числа та рядка еквівалентне конкатенації цих двох, тому результатом буде рядок.

Тепер збережіть код та запустіть додаток; ви побачите повідомлення "My age is 27" у вікні консолі.

Оператори

Після того, як ми оголосили та присвоїли значення змінній, ми можемо використовувати оператори для зміни або комбінування цієї змінної з іншими змінними. У C# є різні типи операторів, включаючи арифметичні, оператори присвоєння, оператори порівняння та логічні оператори.

Арифметичні оператори використовуються для виконання арифметичних операцій, включаючи додавання, віднімання, множення або ділення. Поширені арифметичні оператори: +, -, *, /, або % (модуль).

```
int number1 = 1;// оголошуємо змінну number1
int number2 = 1;// оголошуємо змінну number2
int sum = number1 + number2;// додаємо два числа
int sub = number1 - number2;// віднімаємо два числа
```

Оператори присвоєння використовуються для присвоєння значення змінній: =, +=, -=, *=, /= або %=.

```
int number1 = 1;
number1+=1; //те саме, що number1 = number1 + 1;
number1-=1; //те саме, що number1 = number1 - 1;
number1*=1; //те саме, що number1 = number1 * 1;
number1/=1; //те саме, що number1 = number1 / 1;
```

Оператори порівняння часто використовуються для умов порівняння двох значень: ==, !=, >, <, >= та <=.

```
if (number1 == number2); //якщо number1 дорівнює number2
if (number1 != number2); //якщо number1 та number2 мають різні значення
if (number1 > number2); //якщо number1 більше за number2
if (number1 >= number2); //якщо number1 більше або дорівнює number2
if (number1 < number2); //якщо number1 менше за number2
if (number1 <= number2); //якщо number1 менше або дорівнює number2
```

Умовні оператори

Хоча ми розглянули прості змінні, такі як числа або текст, та як їх змінювати за допомогою операторів, тепер ми розглянемо умовні оператори.

Умовні оператори — це оператори, які виконуються на основі умови. Їхній синтаксис зазвичай такий:

```
if (condition) statement;
```

Це означає: якщо умова підтверджується (є істинною), тоді (і тільки тоді) оператор виконується.

Наприклад, набравши if (a == b), ми маємо на увазі «якщо правда, що a дорівнює b». Аналогічно, якщо ми набираємо if (a>=b), ми маємо на увазі «якщо правда, що a дорівнює або більше за b».

Ми також можемо комбінувати умови. Наприклад, набравши if (a == b && c == 2), ми маємо на увазі «якщо a дорівнює b І c дорівнює 2». Оператор && означає І (AND), і обидві умови повинні бути істинними.

Наприклад, ми могли б перекласти речення «якщо погода сонячна і швидкість вітру менше 5 км/год, то я піду під вітрилами» наступним чином:

```
if (weatherIsSunny == true && windSpeed < 5) IGoSailing = true;
```

При створенні умов ми можемо використовувати оператор АБО (OR), що позначається ||:

```
if (weatherIsTooHot == true || windSpeed > 5) IGoSailing = false;
```

Ще один приклад:

```
if (myName == "Patrick") Console.WriteLine("Hello Patrick");
else Console.WriteLine("Hello Stranger");
```

У попередньому коді ми відображаємо текст "Hello Patrick", якщо змінна myName дорівнює "Patrick"; інакше ми відображаємо повідомлення "Hello Stranger".

Коли ми оцінюємо умови істинності та хибності, ми фактично застосовуємо булеву логіку. Булева логіка працює з булевими числами, які мають два значення: 1 і 0 (або true та false). Оператор ! означає НЕ (NOT) або «протилежне від». Це означає, що якщо змінна має значення true, її протилежність буде false, і навпаки.

Давайте поекспериментуємо з умовними операторами. Додайте наступний код:

```
Console.WriteLine("Please enter your name: ");
string nameOfUser = Console.ReadLine();
if (nameOfUser == "Patrick") Console.WriteLine("Hello Patrick");
else Console.WriteLine("Hello Stranger");
```

У попередньому коді:

- Ми пишемо повідомлення, що просить користувача ввести ім'я.
- Ми оголошуємо змінну nameOfUser, яка містить відповідь користувача.
- Ми зберігаємо ім'я за допомогою методу Console.ReadLine, який читає текст, введений користувачем.
- Ми виконуємо умовний оператор на основі значення змінної nameOfUser.

Тепер спробуємо з числами:

```
Console.WriteLine("What is 1 + 1");
string resultAsString = Console.ReadLine();
int result = Int32.Parse(resultAsString);
if (result == 2) Console.WriteLine("Well done");
else Console.WriteLine("Sorry, wrong answer");
```

У попередньому коді ми конвертуємо рядок відповіді у ціле число за допомогою Int32.Parse, а потім порівнюємо з числом 2.

Методи

Тепер, коли ви можете оголошувати та комбінувати числа та рядки, настав час розглянути методи.

Методи можна порівняти з другом або колегою, якого ви ввічливо просите виконати завдання на основі конкретних інструкцій і повернути інформацію після цього, якщо потрібно.

Наприклад, ви могли б попросити свого друга: "Чи можеш ти, будь ласка, сказати мені, коли я буду святкувати своє 20-річчя, враховуючи, що я народився у 2000 році". Ви даєте другові інформацію (дату народження), і він/вона обчислює рік вашого 20-річчя та повертає цю інформацію. Методи працюють точно так само.

У термінах програмування, метод — це блок інструкцій, який виконує набір дій. Він виконується, коли його викликають з додатку або коли відбувається подія.

Методи дуже корисні, тому що одного разу створений код методу може бути викликаний кілька разів без необхідності переписувати той самий код. Метод може виконувати одну або всі наступні дії:

- Виконувати дію.
- Повертати результат.
- Приймати параметри та обробляти їх.

Метод зазвичай оголошується наступним чином:

```
типДоступу типДанихЩоПовертаються назваМетоду (списокПараметрів)
{
    Виконуйте дії тут
}
```

У попередньому шаблоні:

- Тип доступу визначає область видимості методу, тобто звідки цей метод буде доступний.
- Тип даних, що повертаються, може бути: void (дані не повертаються) або будь-який тип даних, який цей метод може повернути (наприклад, int, bool або string).
- Далі йде назва методу.
- Список параметрів може бути порожнім або містити параметри, розділені комою.

Наприклад:

```
public void myMethod ()
{
    Console.WriteLine("hello");
}
```

Цей метод оголошено як public, його тип повернення — void (він не повертає жодних даних), він не приймає параметрів, і при виклику він виведе повідомлення "hello" в консоль.

Метод викликається за допомогою оператора ():

```
nameOfTheFunction1();
nameOfTheFunction2(value);
int test = nameOfFunction3(value);
```

Давайте створимо наш власний метод. Додайте наступний код до методу Main:

```
int myAge = calculateAge(1990);
Console.WriteLine("My Age is " + myAge);
```

Тепер створимо метод calculateAge після методу Main:

```
public static int calculateAge(int yearOfBirth)
{
    return (2018 - yearOfBirth);
}
```

У попередньому коді метод calculateAge оголошено; він приймає один параметр yearOfBirth. Метод повертає результат віднімання 2018 - yearOfBirth. Коли метод викликається, yearOfBirth дорівнює 1990, тому він поверне 28.

Зверніть увагу, що метод calculateAge оголошено як static, оскільки метод Main за замовчуванням є static, і він може викликати лише статичний метод.

Метод Main є static, оскільки це точка входу програми і це перший метод, який викликається при запуску додатку. Він public, оскільки повинен бути доступний ззовні. Його тип повернення — void, оскільки він зазвичай не повертає дані.

Область видимості змінних

Щоразу, коли ви створюєте змінні в C#, вам потрібно знати про їхню область видимості, щоб використовувати їх там, де це можливо. Це особливо важливо, якщо ви використовуєте методи.

Область видимості змінної визначає, де ви можете використовувати цю змінну в додатку. У C# змінні можуть бути або змінними-членами (member variables), або локальними змінними (local variables).

Змінні-члени можуть використовуватися в конкретному класі або по всьому додатку, залежно від типу доступу. Вони оголошуються на початку класу:

```
class MainClass
{
    public int myVar;
    public void method1()
    {
        myVar = 0;
    }
    public void method2()
    {
        myVar = myVar + 1;
    }
}
```

У попередньому коді змінна-член myVar оголошена на початку класу MainClass; вона потім ініціалізується в method1 та оновлюється в method2. Змінну оголошено як public, що означає, що вона доступна як по всьому класу MainClass, так і ззовні.

Локальні змінні оголошуються (і повинні використовуватися) лише в межах конкретного методу:

```
public void method1()
{
    int myVar;
    myVar = 0;
}
public void method2()
{
    int myVar2;
    myVar2 = 2;
}
```

У попередньому коді myVar є локальною змінною методу method1 і може використовуватися лише в цьому методі; myVar2 є локальною змінною методу method2.

Зверніть увагу, що завжди добре уникати так званого затінення змінних (variable shadowing), яке відбувається, коли ви даєте локальній змінній таку ж назву, як і змінній-члену, що може призвести до плутанини.

Декілька речей, які слід пам'ятати при створенні додатків (контрольний список)

- Кожна відкриваюча дужка має відповідну закриваючу дужку.
- Всі змінні записуються послідовно (правопис та регістр). Назва кожної змінної чутлива до регістру.
- По можливості оголошуйте змінну перед її використанням.
- Для кожної назви змінної всі слова (крім першого) мають велику початкову літеру (camelCase).
- Для кожного методу всі слова (крім першого) мають велику початкову літеру.
- Усі оператори закінчуються крапкою з комою.
- Для операторів if умова знаходиться в круглих дужках.
- Для операторів if умова використовує синтаксис "==" замість "=".
- При виклику методу використовується точна назва цього методу (з урахуванням регістру).
- При зверненні до змінної враховується її область видимості.
- Локальні змінні оголошуються та можуть використовуватися в межах одного методу.
- Змінні-члени оголошуються поза методами та можуть використовуватися в будь-якому місці класу.

Найкращі практики

Іменування змінних:

- Використовуйте зрозумілі назви, які ви зможете зрозуміти навіть через два тижні.

```
string myName = "Patrick";//ДОБРЕ
string b = "Patrick";//НЕ ДУЖЕ ДОБРЕ
```

- Пишіть з великої літери початок усіх слів у назві, крім першого (camelCase).

```
bool testIfTheNameIsCorrect = true;// ДОБРЕ
bool testifthenameiscorrect = true; // НЕ ДУЖЕ ДОБРЕ
```

Методи:

- Використовуйте унікальні (різні) назви.
- Перевіряйте, що всі відкриваючі дужки мають відповідні закриваючі дужки.
- Робіть відступи у своєму коді.
- Коментуйте свій код якомога більше, щоб пояснити, як він працює.

Підсумок розділу

У цьому розділі нам вдалося створити ваші найперші додатки. Ви стали більш комфортними зі створенням змінних та методів, а також з відображенням інформації через вікно консолі. Нарешті, ви також навчилися використовувати умовні оператори, щоб обробка могла виконуватися на основі конкретних умов. Спираючись на ці знання, ви створили прості програми, де користувач вводить інформацію, а програма обчислює та виводить результат. Отже, ми пройшли значний шлях від відсутності знань C# до створення вашого першого додатку: молодці!

Вікторина

Настав час перевірити свої знання. Будь ласка, вкажіть, чи є наступні твердження ПРАВИЛЬНИМИ чи НЕПРАВИЛЬНИМИ.

1. Основною точкою входу в програму C# є метод під назвою Main.
2. Метод Main зазвичай є public, щоб його можна було викликати ззовні вашого додатку.
3. Метод Main зазвичай оголошується як static.
4. Команда Console.WriteLine відображає інформацію у вікні консолі.
5. Console.ReadLine записує інформацію, введену користувачем у вікні консолі.
6. У C# текстові значення оточуються подвійними лапками.
7. У C# локальна змінна може використовуватися лише в методі, де вона була спочатку визначена.
8. У C# змінна-член може використовуватися по всьому класу, де вона була оголошена.
9. У C# метод завжди повертає значення.
10. У C# закоментований код не виконується.

Відповіді на вікторину

1. ПРАВИЛЬНО. 2. ПРАВИЛЬНО. 3. ПРАВИЛЬНО. 4. ПРАВИЛЬНО. 5. ПРАВИЛЬНО. 6. ПРАВИЛЬНО. 7. ПРАВИЛЬНО. 8. ПРАВИЛЬНО. 9. НЕПРАВИЛЬНО. 10. ПРАВИЛЬНО.

Контрольний список

Ви можете перейти до наступного розділу, якщо можете зробити наступне:

- Створювати змінні.
- Комбінувати змінні.
- Створювати метод.
- Викликати метод.

Завдання

Завдання 1

Створіть метод під назвою calculatePrice, який робить наступне:

- Приймає два параметри: nbItems та unitPrice.
- Множить ці два числа та зберігає результат у змінній finalPrice.
- Повертає значення змінної finalPrice.

Після створення методу викличте його з двома значеннями на ваш вибір та відобразіть результат у вікні консолі.

Завдання 2

Створіть метод, який робить наступне:

- Запитує у користувача його/її ім'я.
- Запитує у користувача його/її прізвище.
- Відображає повне ім'я (тобто ім'я, за яким слідує пробіл, за яким слідує прізвище) у вікні консолі.

Після створення методу викличте його.

2. Подяка

Я хотів би подякувати вам за завершення цієї книги. Ця книга охопила лише основи C#, і щоб дізнатися більше, вам потрібно завантажити книгу «C# Programming from Zero to Proficiency (Beginner)», щоб вивчити ще більше навичок програмування на C#.

Щоб книгу можна було постійно покращувати, я дуже ціную ваші відгуки. Будь ласка, залиште корисний відгук та надішліть мені електронного листа (learncsharpforbeginners@gmail.com) з будь-якими пропозиціями. Я читаю та відповідаю на кожен лист.

Дякую!